



EMERITUS

Learn. From the world's best.

Deep Feedforward Neural Networks - I



EMERITUS

Learn. From the world's best.

Image Classification with Keras

Hands-on Workshop Using Tensorflow And Keras



Build image classifiers with
TensorFlow & Keras

Move from simple models to deep
networks

Explore and apply optimisation
techniques

Environment & Dataset Setup

Tools And Datasets For Image Classification

Required Tools

- Python 3.10+
- TensorFlow 2.x (Keras)
- NumPy
- Matplotlib

```
# Install dependencies  
!pip install -U tensorflow matplotlib  
import tensorflow as tf  
import numpy as np  
import matplotlib.pyplot as plt  
tf.random.set_seed(42)np.random.seed(42)
```

Dataset Options

We'll work with standardised image datasets:

- **Fashion-MNIST:** 28×28 grayscale clothing images in 10 classes
- **CIFAR-10:** 32×32 RGB object images in 10 categories

Baseline Model: Softmax on Flattened Pixels

Start with a simple softmax classifier on raw pixels to set a baseline and check linear separability

```
from tensorflow.keras import Sequential, layers
import tensorflow as tf

def baselinelogit(input_shape, num_classes):
    return Sequential([
        layers.Input(shape=input_shape),
        layers.Flatten(),
        layers.Dense(num_classes, activation="softmax")
    ])

num_classes = 10
input_shape = x_tr.shape[1:]
```

Baseline Model: Softmax on Flattened Pixels

```
m_base = baselinelogit(input_shape, num_classes)

m_base.compile(
    optimiser=tf.keras.optimisers.SGD(0.1),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

h_base = m_base.fit(x_tr, y_tr, validation_data=(x_val, y_val), epochs=10, batch_size=256, verbose=0)

print("Baseline (val):", m_base.evaluate(x_val, y_val, verbose=0))
```

Baseline Model: Softmax on Flattened Pixels



Input Image



Flattened Pixels



Dense Layer



Linear Scores



Softmax Output

This simple model has one trainable layer, treating each pixel as an independent feature and ignoring spatial relationships

Deep Feedforward Network (MLP)

Upgrade to an MLP with hidden layers and ReLU, capturing complex non-linear pixel patterns while still ignoring spatial structure

```
import tensorflow as tf
from tensorflow.keras import Sequential, layers

def mlp(input_shape, num_classes, width=512,
        depth=2, l2=0.0, dropout=0.0,
        init="he_uniform"):
    seq = [
        layers.Input(shape=input_shape),
        layers.Flatten()
    ]

    for _ in range(depth):
        seq.append(
            layers.Dense(
                width,
                activation="relu",
                kernel_regularizer=tf.keras.regularizers.L2(l2) if l2 > 0 else None,
                kernel_initializer=init
```


Deep Feedforward Network (MLP)

```
)  
    )  
    if dropout > 0:  
        seq.append(layers.Dropout(dropout))  
  
    seq.append(layers.Dense(num_classes, activation="softmax"))  
    return Sequential(seq)  
  
# Model initialisation  
m_mlp = mlp(input_shape, num_classes, width=512, depth=2)  
  
# Compile model  
m_mlp.compile(  
    optimizer=tf.keras.optimisers.Adam(1e-3),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

MLP Parameters

1. Width: 512 neurons per hidden layer

2. Depth: 2 hidden layers

3 Optimiser: Adam, learning rate 0.001

Optimiser Bake-Off

Comparing Training Performance

Different optimisers affect training speed and performance. Here we compare four using the same MLP

1

SGD

- LR 0.1
- Simple, may converge slowly

2

Momentum

- LR 0.05, momentum 0.9
- Speeds up by adding velocity

3

RMSProp

- LR 0.001
- Adaptive, scales by recent gradients

4

Adam

- LR 0.001
- Combines Momentum + RMSProp, works well broadly

Optimiser Bake-Off

```
import tensorflow as tf

# Define optimisers
opts = {
    "sgd": tf.keras.optimisers.SGD(0.1),
    "mom": tf.keras.optimisers.SGD(0.05, momentum=0.9),
    "rms": tf.keras.optimisers.RMSprop(1e-3),
    "adam": tf.keras.optimisers.Adam(1e-3),
}

# Training function with given optimiser
def train_with_opt(opt, label):
    m = mlp(input_shape, num_classes, width=512, depth=2)
    m.compile(
        optimiser=opt,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
```

Optimiser Bake-Off

```
h = m.fit(  
    x_tr, y_tr,  
    validation_data=(x_val, y_val),  
    epochs=10,  
    batch_size=256,  
    verbose=0  
)  
val_acc = m.evaluate(x_val, y_val, verbose=0)[1]  
print(f"{label}: val_acc={val_acc:.3f}")  
return h
```

```
# Train with each optimizer and store histories  
histories = {name: train_with_opt(opt, name) for name, opt in opts.items ()}
```

We'll analyse the training curves for each optimizer in the visualisation section later

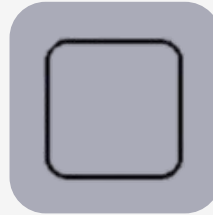
Learning Rate Scheduling and Early Stopping

Adaptive Training Control

Learning rate scheduling adjusts step size to improve convergence, while early stopping prevents overfitting



Monitor Validation Loss:
Track performance each epoch



Reduce Learning Rate:
Halve if loss stalls for 2 epochs



Stop Training
End after 3 stagnant epochs, restore best weights

Learning Rate Scheduling and Early Stopping

Adaptive Training Control

```
import tensorflow as tf

# Define callbacks
cb = [tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, verbose=1),
      tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)]

# Build model
m_sched = mlp(input_shape, num_classes, width=512, depth=3)

# Compile model
m_sched.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)
```

Learning Rate Scheduling and Early Stopping

Adaptive Training Control

```
# Train model with callbacks
h_sched = m_sched.fit(x_tr, y_tr, validation_data=(x_val, y_val),
    epochs=30,
    batch_size=256,
    callbacks=cb,
    verbose=0
)

# Evaluate model
print("Schedule+ES (val):", m_sched.evaluate(x_val, y_val, verbose=0))
```

Regularisation: L2

Controlling Model Complexity

L2 Regularisation

Penalises large weights by adding squared values to the loss function:

- Encourages smaller, more balanced weights
- Reduces overfitting to training data
- Parameter: $l2 = 1e-4$ (strength)

```
m_reg = mlp(input_shape, num_classes, width=512, depth=3, l2=1e-4,
dropout=0.3)m_reg.compile( optimiser=tf.keras.optimisers.Adam(1e-3),
loss="sparse_categorical_crossentropy", metrics=["accuracy"])h_reg = m_reg.fit( x_tr, y_tr,
validation_data=(x_val, y_val), epochs=20, batch_size=256, verbose=0)print("Reg (val):",
m_reg.evaluate(x_val, y_val, verbose=0))
```


Visualising Training Progress

Tracking Model Behaviour

Plotting loss and accuracy shows training behaviour and reveals overfitting, underfitting, or optimisation issues



Healthy Learning:
Training and validation
improve together



Overfitting: Training
improves while
validation worsens



Underfitting: Both
plateau below best
performance

Visualising Training Progress

Tracking Model Behaviour

```
import matplotlib.pyplot as plt

def plot_hist(h, title):
    plt.figure(figsize=(8, 5))

    for k in ("loss", "val_loss", "accuracy", "val_accuracy"):
        if k in h.history:
            plt.plot(h.history[k], label=k)

    plt.title(title)
    plt.xlabel("Epochs")
    plt.ylabel("Value")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Test Set Evaluation and Confusion Matrix

Measuring Final Model Performance

The final step is testing on a held-out set to get an unbiased measure of overall and class-specific performance

```
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
def evaluate_and_report(model, X, y, name):
    # Evaluate model
    test_loss, test_acc = model.evaluate(X, y, verbose=0)
    # Predictions
    y_pred = np.argmax(model.predict(X, verbose=0), axis=1)
    # Print results
    print(f"{name} — Test acc: {test_acc:.3f}")
    print("Confusion Matrix:")
    print(confusion_matrix(y, y_pred))
    print("\nClassification Report:")
    print(classification_report(y, y_pred, digits=3))
    # Evaluate the chosen best-performing model
    evaluate_and_report(m_sched, x_te, y_te, "Best Model")
```

Test Set Evaluation and Confusion Matrix

Measuring Final Model Performance

Interpreting the Confusion Matrix

Rows = true classes, columns = predicted classes. Diagonal = correct predictions, off-diagonal = errors.

The classification report gives precision, recall, and F1-score, highlighting class imbalances and weaknesses

In-Class Lab Exercises

Your Turn to Experiment

Now it's your turn to experiment! Complete the following exercises and analyse your results

A. Baseline vs MLP

- Run both for 10 epochs
- Record validation & test accuracy
- Note over/under-fitting
- Explain why MLP performs better

B. Optimiser Bake-Off

- Create table of metrics per optimiser
- Track time/epoch & best validation accuracy
- Plot learning curves side by side
- Identify best performer and why

In-Class Lab Exercises

Your Turn to Experiment

C. Regularisation Analysis

- Test different L2 & Dropout values
- Compare train vs validation accuracy gap
- Find optimal settings

D. Early Stopping

- Identify best stopping epoch
- Compare with fixed-epoch training
- Measure computational savings

Deliverables

- One comparison table of models
- Two visualisation plots with insights
- Three bullet points on what boosted accuracy most & why

Troubleshooting Cheat Sheet

Quick Fixes for Common Issues

Loss Plateau / NaN Values

- Lower learning rate (e.g. $\times 0.1$)
- Switch to Adam for stability
- Check label type (int for sparse)
- Normalise inputs (0–1 or mean=0, std=1)
- Use gradient clipping

Underfitting (Both Metrics Low)

- Add layers/neurons
- Train longer
- Use momentum/adaptive optimisers
- Reduce regularisation
- Try complex models (e.g. CNNs for images)

Troubleshooting Cheat Sheet

Quick Fixes for Common Issues

Overfitting (Train » Val)

- Add dropout (0.2–0.5)
- Apply L2 regularisation ($1e-5$ – $1e-3$)
- Reduce model size
- Use early stopping (patience 3–5)
- Augment training data

Unstable Learning Curves

- Apply LR schedules (ReduceOnPlateau)
- Add gradient clipping
- Increase batch size
- Try different weight inits
- Use label smoothing (0.1)



Demo

Q&A Session



Open floor for questions and clarifications.